Лекция 4. Масштабируемая аналитика и машинное обучение в Spark

Воспроизводимый ML-pipeline над большими наборами данных

Курс «Технологии и инструментарий анализа больших данных»



Раздел

Введение



Тема четвёртой лекции

Сегодня мы переходим от распределённой обработки данных к распределённому машинному обучению.

Главная идея

В практическом Big Data-проекте важен не отдельный алгоритм сам по себе, а **воспроизводимый конвейер** подготовки данных, построения признаков, обучения, оценки качества и применения модели к новым данным.

На лекции нас интересуют три уровня:

- математическая постановка ML-задачи;
- инженерная организация вычислений в Spark ML;
- вычислительная стоимость обучения, валидации и подбора гиперпараметров.



План лекции

1. ML как слой потока Big Data.
2. Постановки регрессии, классификации, кластеризации.
3. Разделение данных и проблема утечки данных.
4. Инженерия и вектор признаков.
5. Spark ML: `Transformer`, `Estimator`, `Model`, `Pipeline`.
6. Базовые модели ML.
7. Метрики качества классификации.
8. Деревья решений, bagging, random forest, boosting.
9. Кластеризация и K-Means.
10. Кросс-валидация, настройка гиперпараметров и масштабируемость ML.



Почему это не просто «ML на большом CSV»

Обычный учебный ML

- данные помещаются в память;
- один процесс Python;
- эксперимент вручную;
- модель обучается быстро;
- результат легко пересчитать.

ML в Big Data

- данные распределены по партициям;
- обучение требует перемешивания и кэш;
- признаки строятся конвейером;
- оценка качества дорогая;
- эксперимент должен быть воспроизводимым.

Переход к Spark ML меняет не формальное определение модели, а **способ выполнения вычислений** и дисциплину работы с данными.

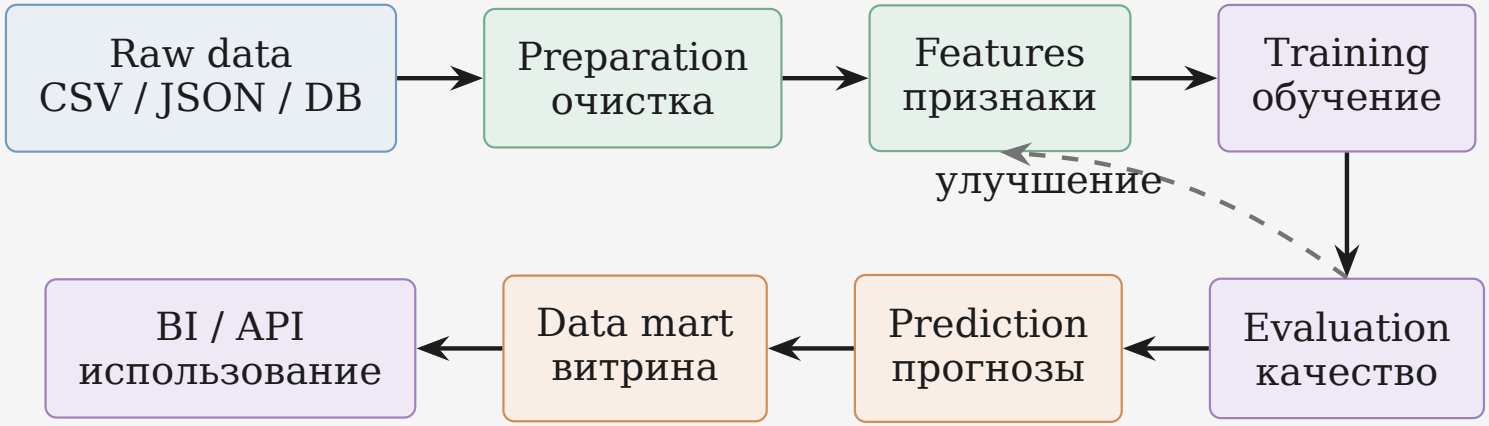


Раздел

ML в Big Data pipeline



Место ML в аналитической системе



Определение

Поток машинного обучения — это упорядоченная последовательность преобразований данных и этапов обучения, которая переводит сырые данные в модель и применимые предсказания.



Что является результатом ML-слоя

ML-слой редко завершается только файлом модели. В промышленной системе обычно появляются несколько результатов:

- обученная модель с параметрами;
- метрики качества и протокол эксперимента;
- таблица предсказаний для дальнейшей аналитики;
- витрина признаков или витрина прогнозов;
- правила повторного запуска потока на новых данных.

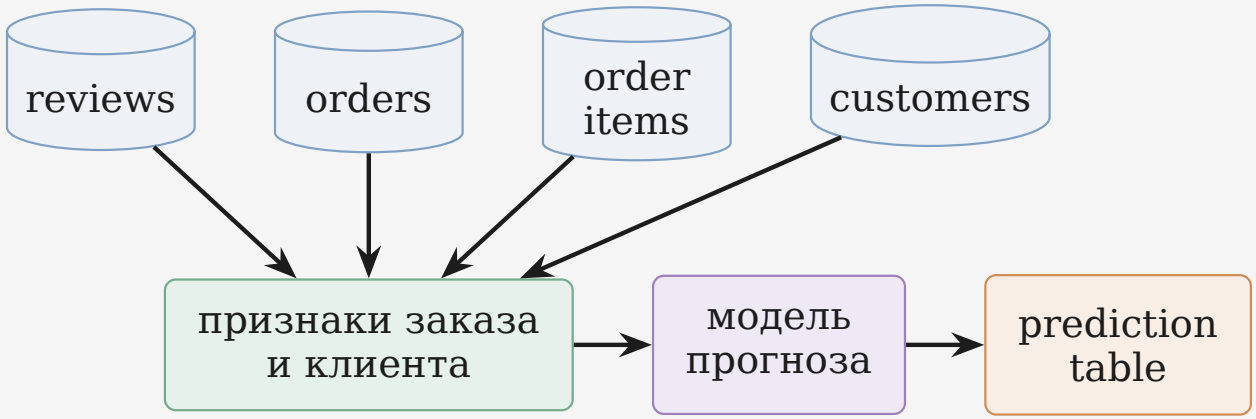
Практический критерий

Модель считается частью системы только тогда, когда можно воспроизвести путь от исходных данных до предсказаний и объяснить, на каком наборе данных оценивалось качество.



Пример. Электронная коммерция

Для курса удобно мыслить в терминах интернет-магазина.



Возможные задачи: прогноз стоимости заказа, выявление негативного отзыва, сегментация клиентов, оценка вероятности повторной покупки.



Почему ML становится вычислительной задачей

Пусть исходный набор данных содержит N объектов и p признаков. Даже простая линейная модель требует многократного прохода по данным:

$$X \in \mathbb{R}^{N \times p}, \qquad \theta_{t+1} = \theta_t - \eta \nabla_\theta L(X, y; \theta_t).$$

Если данные не помещаются в память одного узла, то каждый шаг обучения становится распределённой операцией.

Инженерный смысл

Стоимость обучения определяется не только числом арифметических операций, но и чтением данных, сериализацией, перемешиванием, сетевыми передачами, кэшированием и повторными проходами по `DataFrame`.



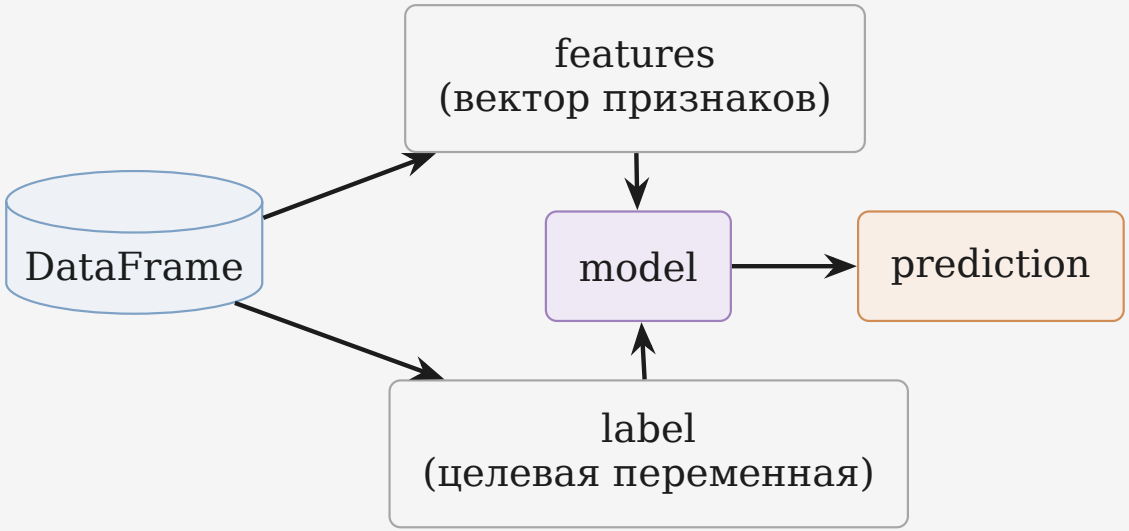
Данные, признаки, целевая переменная

Базовая формализация

Объект описывается вектором x_i, целевое значение — y_i.
Обучение ищет отображение

$$f : X \to Y,$$

которое хорошо работает не только на обучающей выборке, но и на новых объектах.





Раздел

Основные постановки машинного обучения



Три базовые постановки

Регрессия

$$f : X \to \mathbb{R}$$

Прогноз числовой величины: цена, время доставки, спрос.

Классификация

$$f : X \to \{C_1, \dots, C_k\}$$

Выбор класса: 1 / 0, + / -.

Кластеризация

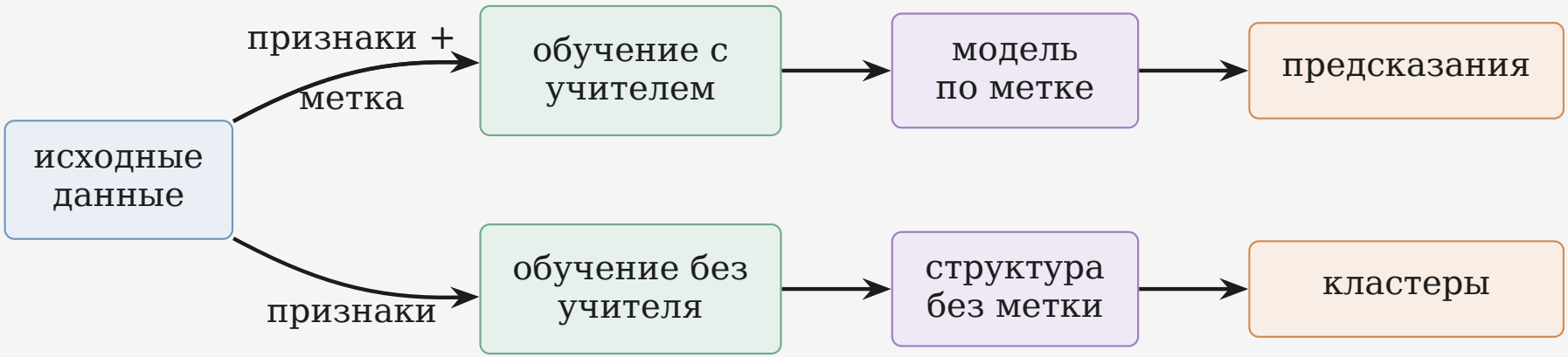
$$X \to \{K_1, \dots, K_m\}$$

Поиск групп без заранее заданной метки.

В Spark ML все эти задачи выражаются через похожий программный интерфейс, но различаются функцией потерь и метриками качества.



Обучение с учителем и без учителя



Ключевое отличие

В обучении с учителем качество измеряется сравнением предсказаний с известными ответами. В обучении без учителя качество часто ***оценивается косвенно*** через компактность, разделимость или бизнес-интерпретируемость групп.

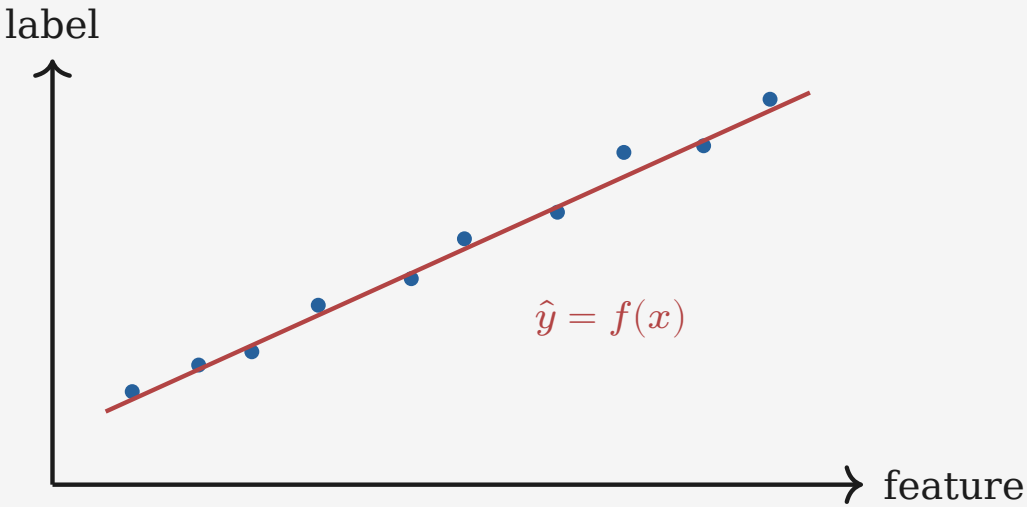


Регрессия. Прогноз числовой величины

В задаче регрессии целевая переменная имеет числовую природу:

$$y_i \in \mathbb{R}.$$

Пример: прогноз общей стоимости заказа по числу товаров, категории, региону, способу оплаты и истории клиента.

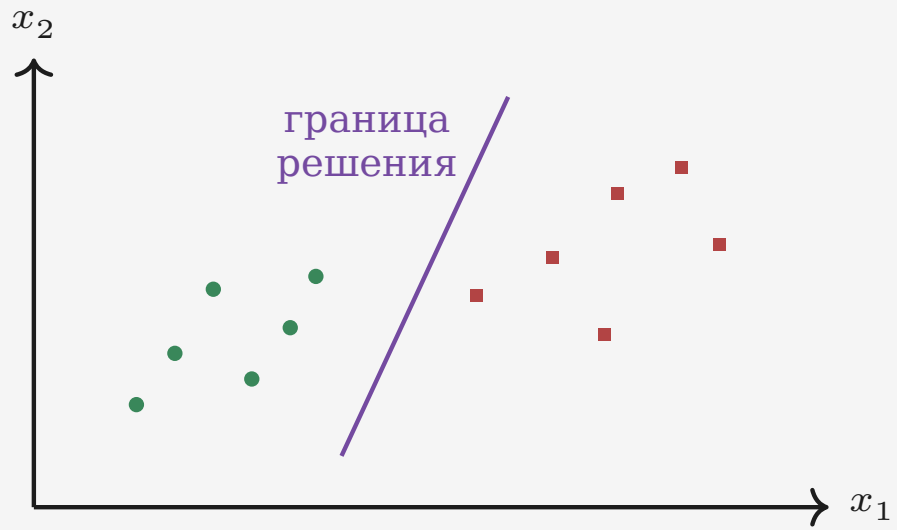


Для регрессии естественны метрики ошибки: MSE, RMSE, MAE, R^2.

Классификация. Выбор категории

В задаче классификации целевая переменная дискретна:

$$y_i \in \{C_1, C_2, \dots, C_k\}.$$

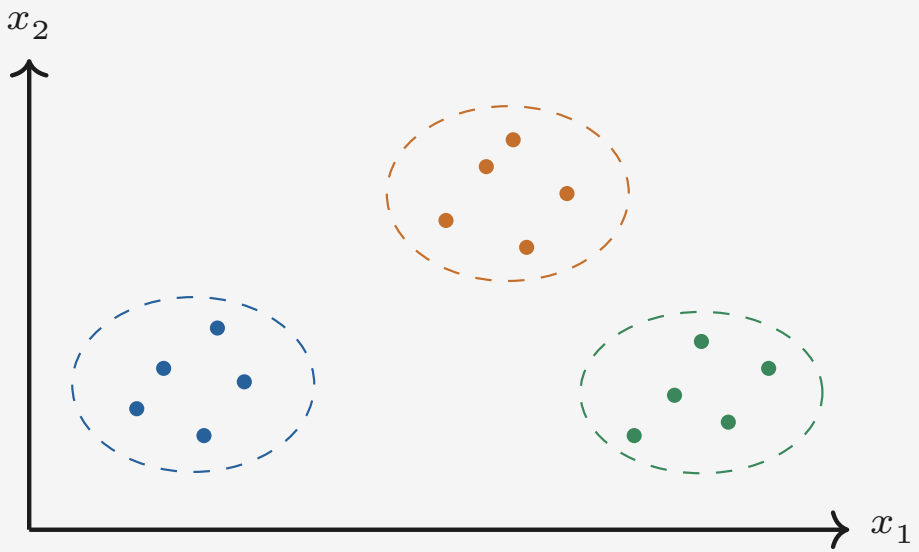


Задача модели — построить границу решений или вероятностное правило, которое переводит признаки объекта в класс.

Кластеризация. Структура без ответа

Определение

Кластеризация — это разбиение множества объектов на группы так, чтобы объекты внутри одной группы были похожи друг на друга сильнее, чем на объекты из других групп.



Кластеризация полезна для сегментации клиентов, группировки товаров, анализа поведения и поиска аномальных групп.



Общая математическая форма обучения

Для многих моделей обучения с учителем процесс настройки параметров можно записать как задачу минимизации функции потерь:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(x_i; \theta)) + \lambda R(\theta).$$

- L — ошибка модели на одном объекте;
- $R(\theta)$ — регуляризация, ограничивающая сложность модели;
- λ — сила регуляризации; θ — обучаемые параметры.

Смысл для Spark

Когда N велико, сумма по объектам естественно раскладывается по партициям. Но агрегация градиентов, статистик или критериев разбиения требует синхронизации между исполнителями.



Раздел

Разделение данных и утечка данных



Тренировка, валидация, тест

Правило разделения

Данные для машинного обучения делятся минимум на обучающую и тестовую части; для выбора модели или гиперпараметров вводят также валидационный набор:

$$D = D_{train} \cup D_{val} \cup D_{test}, \qquad D_i \cap D_j = \emptyset.$$



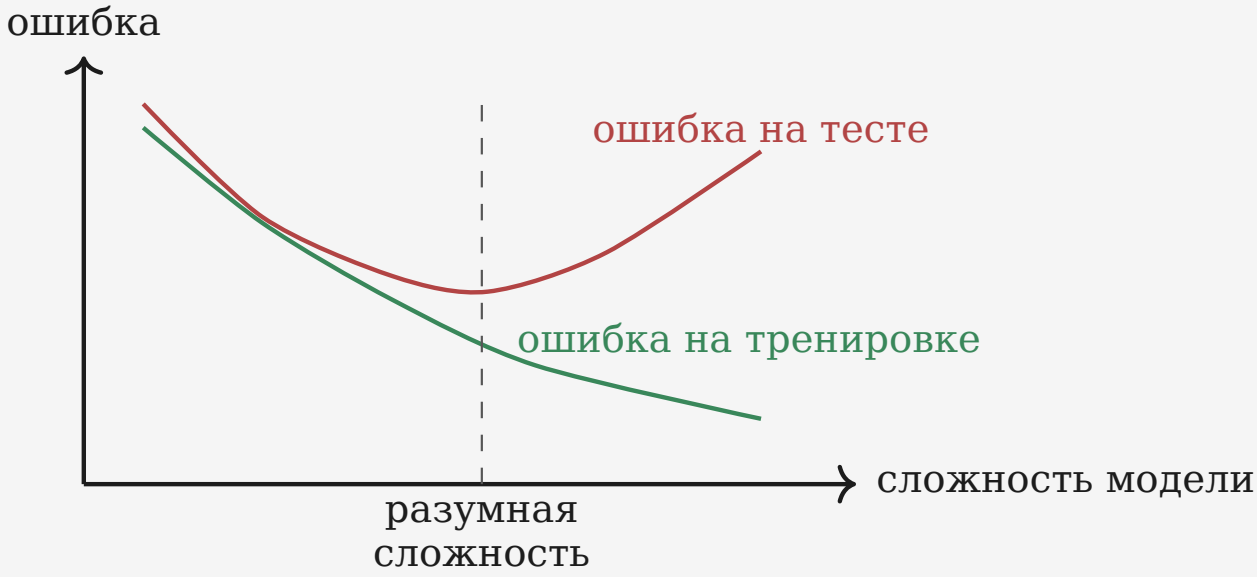
модель не должна получать информацию из будущей оценки качества

Валидационный набор используется для выбора решения; тестовый набор — для финальной честной оценки.



Почему нельзя оценивать качество на тренировочном наборе

Если модель проверяется на тех же данных, на которых обучалась, мы измеряем не способность обобщать, а способность подстроиться под обучающую выборку.



Переобучение

Переобучение возникает, когда модель хорошо описывает обучающие данные, но плохо работает на новых объектах.

Утечка данных

Определение

Утечка данных — это попадание информации из валидации/теста или из будущего в признаки, доступные модели во время обучения.

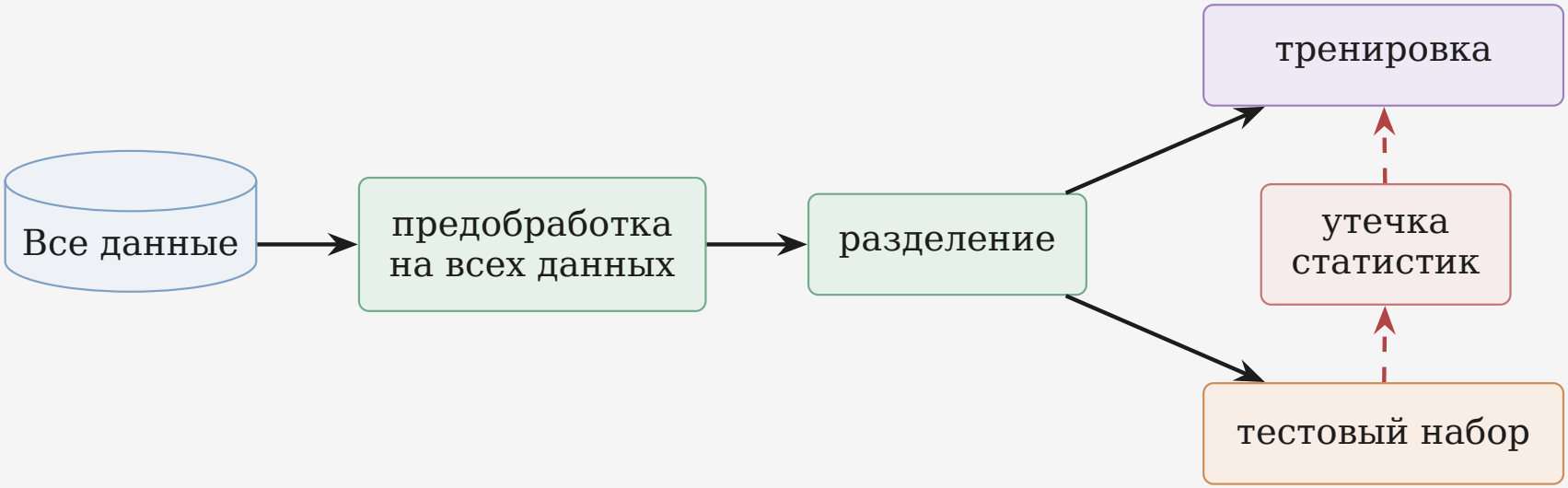
Типичные формы утечки:

- преобразования данных до разделения;
- использование признаков, известных только после события;
- случайное перемешивание временного ряда вместо временного разделения;
- один объект одновременно в тренировочной и тестовой выборке;
- агрегации по всей истории, включая будущий период.

В больших данных утечка данных особенно опасна, так как ошибка может быть не видна синтаксически, но метрики будут неправдоподобно высокими.



Схема утечки при неправильной подготовке признаков



Правильная логика

Сначала выполняется разделение данных. Затем все обучаемые преобразования признаков выполняются только на D_{train}, а к валидации/тесту применяется уже обученное преобразование.

Случайное и временное разделение признаков

Случайное разделение

Подходит, если объекты независимы и порядок наблюдений не несёт причинного смысла.

$$D \xrightarrow{random} D_{train}, D_{test}$$

Временное разделение

Нужно, если модель прогнозирует будущее по прошлому.

$$D_{past} \rightarrow D_{future}$$

В задачах спроса, оттока, задержек доставки, финансовых событий и мониторинга случайное перемешивание может создавать утечку будущего.



Раздел

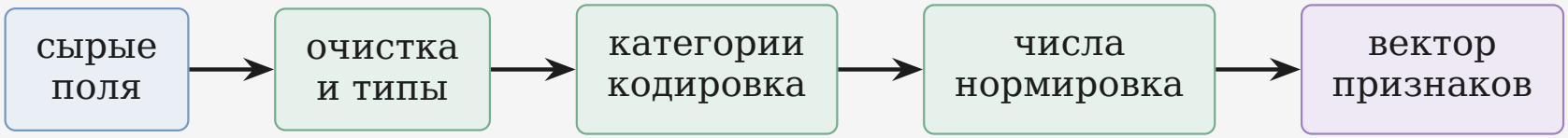
Инженерия признаков



Смысл инженерии признаков

Определение

Инженерия признаков — это построение числового представления объектов, пригодного для обучения модели.



Для Spark ML принципиально, что большинство алгоритмов ожидает таблицу `признаков`, содержащий вектор признаков, и один столбец `меток`.



Пропущенные значения

Пропуски зачастую кодируют особенности процесса сбора данных. Методы по исправлению:

- **Удаление строк**. Просто, но может сместить распределение данных.
- **Заполнение константой**. Полезно для категорий и специальных случаев.
- **Mean/median imputation**. Применимо к числовым признакам.
- **Missing indicator**. Отдельный бинарный признак факта пропуска.

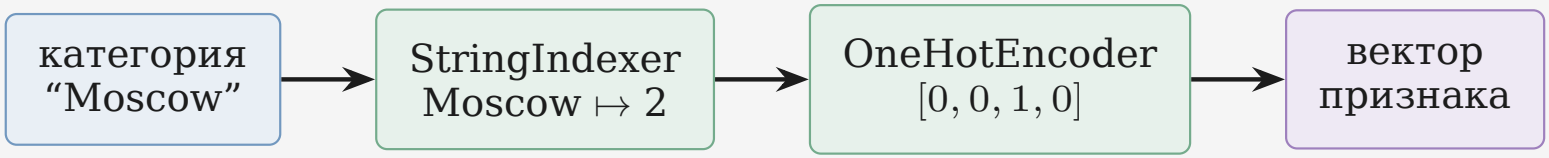
Важно для потока обработки

Если стратегия заполнения оценивает статистики по данным, она должна обучаться только на D_{train}, иначе возникает утечка данных.



Категориальные признаки

Категориальные значения нельзя напрямую передавать большинству численных алгоритмов.



Важно

Индекс категории сам по себе обычно не означает порядок. Поэтому для линейных моделей часто используют one-hot encoding, чтобы не навязать ложное отношение «больше / меньше».



Масштабирование числовых признаков

Для многих методов расстояния и оптимизации масштаб признаков имеет значение.

$$z = \frac{x - \mu}{\sigma}, \qquad x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}.$$

Когда критично

- логистическая регрессия;
- линейная регрессия с регуляризацией;
- K-Means;
- методы на расстояниях.

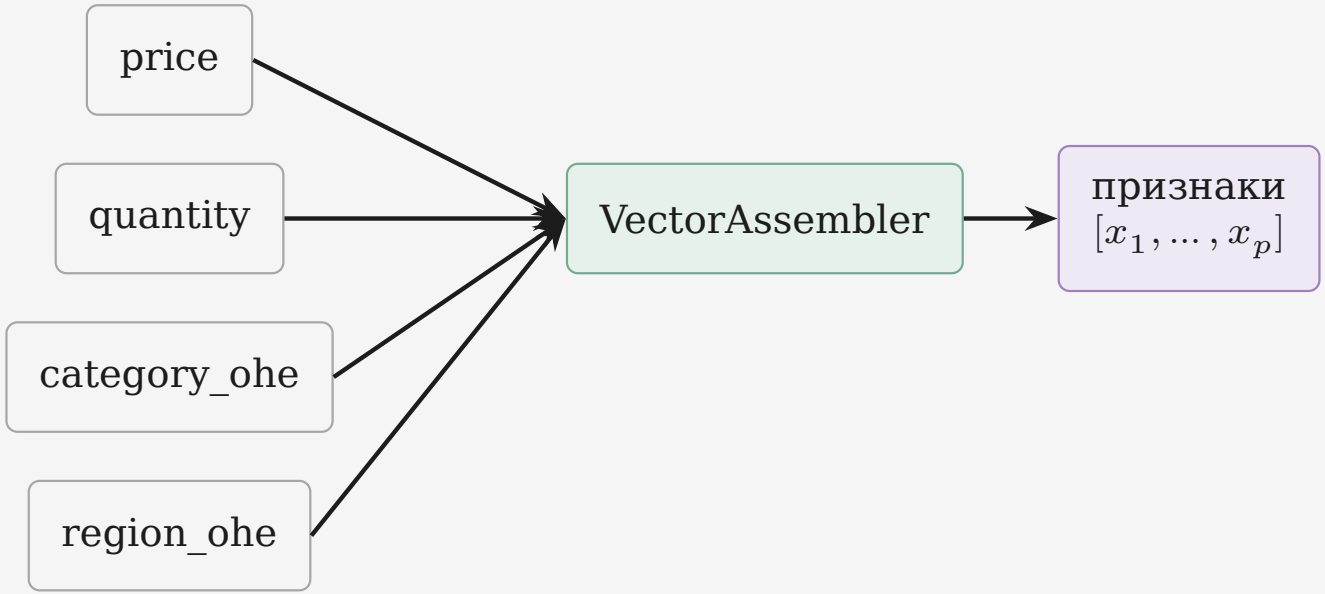
Когда менее критично

- деревья решений;
- случайный лес;
- большинство моделей, основанных на правилах.



VectorAssembler

В Spark ML признаки разных столбцов собираются в один вектор.



Итоговое представление объекта

$$x_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \in \mathbb{R}^p.$$

Именно такие векторы поступают в алгоритм обучения.

Разреженные и плотные представления

После one-hot кодирования или текстовой векторизации признаки часто становятся разреженными:

$$x = (0, 0, 0, 1, 0, 0, 0, 0, 1, 0, ...).$$

Плотный вектор

Хранит все координаты, включая нули. Удобен для малой размерности.

Разреженный вектор

Хранит только ненулевые координаты и их индексы. Критичен для высокой размерности.

В текстовой аналитике разреженные векторы станут главными, так как словарь может содержать десятки и сотни тысяч признаков.



Пример сборки признаков

```
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler

city_indexer = StringIndexer(inputCol="city", outputCol="city_idx")
city_encoder = OneHotEncoder(inputCol="city_idx", outputCol="city_ohe")

assembler = VectorAssembler(
    inputCols=["price", "quantity", "city_ohe"],
    outputCol="features"
)
```

Что важно увидеть

`StringIndexer` и другие обучаемые преобразования имеют метод `fit`. Поэтому они должны быть частью потока обработки, а не случайным предварительным действием над всем набором данных.



Раздел

Поток обработки в Spark ML



Три сущности Spark ML

Transformer

Преобразует один `DataFrame` в другой.

$$DF \to DF'$$

Пример: обученный encoder, scaler, model.

Estimator

Обучается на данных и создаёт `Model`.

$$fit(DF) \to Model$$

Пример: LogisticRegression.

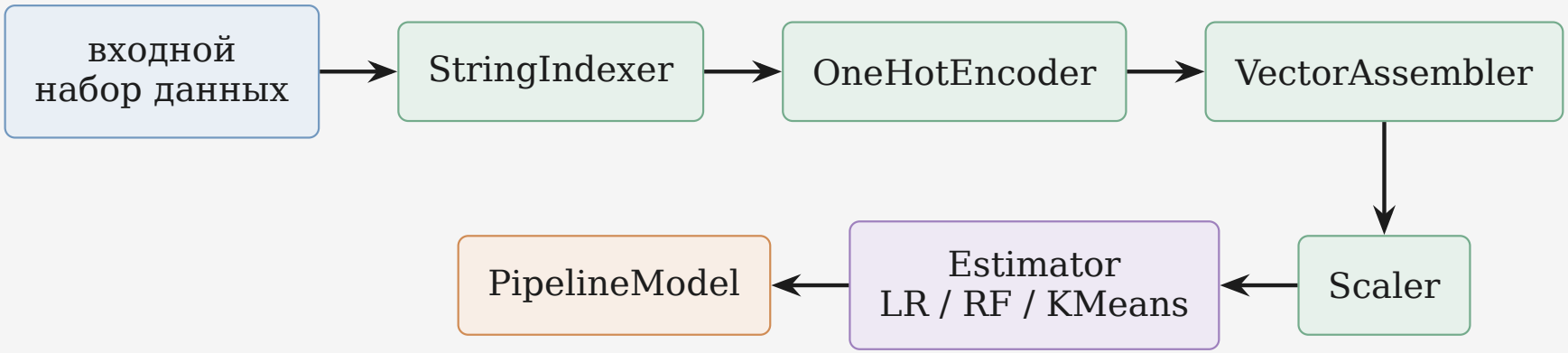
Model

Обученный transformer, применяемый к новым данным.

$$transform(DF) \to DF'$$



Пайплайн как композиция этапов

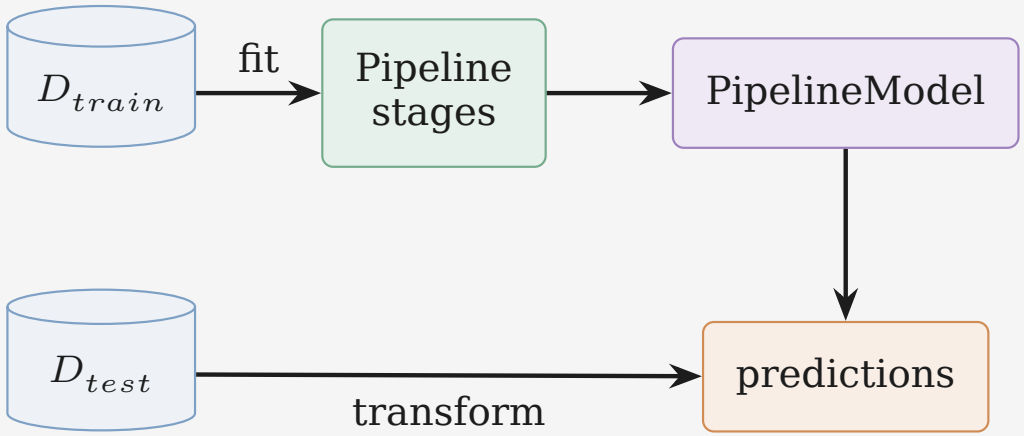


Главное преимущество

Pipeline фиксирует не только алгоритм, но и всю последовательность подготовки признаков. Это повышает воспроизводимость эксперимента и снижает риск несогласованной обработки train и test.



Fit и transform



Правило

Всё, что «узнаёт» статистики из данных, должно обучаться на train. Всё, что оценивает качество, должно применяться к данным, не участвовавшим в `fit`.



Пример Pipeline в PySpark

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import LogisticRegression

indexer = StringIndexer(inputCol="category", outputCol="category_idx")
encoder = OneHotEncoder(inputCol="category_idx", outputCol="category_ohe")
assembler = VectorAssembler(
    inputCols=["price", "quantity", "category_ohe"],
    outputCol="features"
)
lr = LogisticRegression(featuresCol="features", labelCol="label")

pipeline = Pipeline(stages=[indexer, encoder, assembler, lr])
model = pipeline.fit(train_df)
predictions = model.transform(test_df)
```



Pipeline как объект воспроизводимости

Что фиксирует pipeline

- порядок преобразований;
- имена входных и выходных столбцов;
- параметры этапов;
- обученные индексы категорий;
- параметры модели.

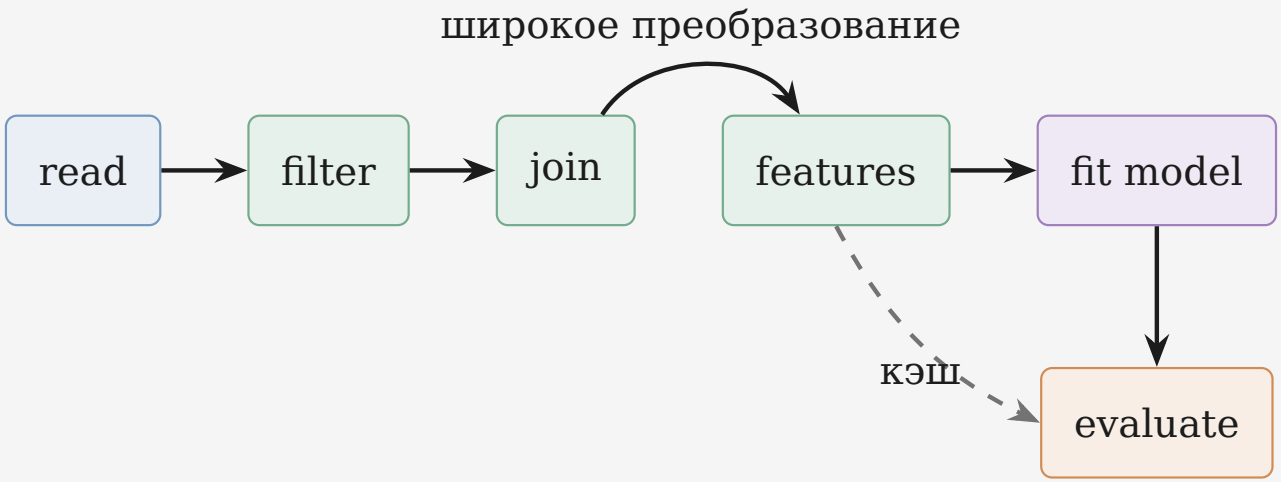
Что надо фиксировать отдельно

- версию данных;
- seed и стратегию split;
- параметры Spark-сессии;
- метрики качества;
- версию кода и библиотек.

Воспроизводимость ML — это свойство всей экспериментальной процедуры, а не только файла модели.



Pipeline и DAG Spark



Ленивое выполнение пайплайна

Pipeline Spark ML не отменяет ленивого выполнения. Этапы построения признаков входят в общий вычислительный план Spark и выполняются только при действии внутри обучения или оценки.



Раздел

Линейная и логистическая регрессия



Линейная регрессия

Модель

$$\hat{y} = w_0 + w_1 x_1 + ... + w_p x_p = w_0 + w^T x.$$

Линейная регрессия прогнозирует числовую величину как линейную комбинацию признаков.

Типичная функция ошибки:

$$MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2.$$

Интерпретация

Коэффициент w_j показывает, как меняется прогноз при изменении признака x_j, если остальные признаки фиксированы. Но такая интерпретация корректна только при осмысленной подготовке признаков.



Регуляризация в линейных моделях

При большом числе признаков модель может переобучаться. Регуляризация добавляет штраф за сложность параметров.

L2 / Ridge

$$R(w) = \|w\|_2^2$$

Сдерживает рост весов, но редко делает их точно нулевыми.

L1 / Lasso

$$R(w) = \|w\|_1$$

Может занулять часть весов и выполнять отбор признаков.

В Spark параметры регуляризации становятся гиперпараметрами, которые можно подбирать через валидацию или кросс-валидацию.



Метрики регрессии

Метрика	Формула	Смысл
MSE	$\frac{1}{N}\sum(y_i - \hat{y}_i)^2$	сильно штрафует крупные ошибки
RMSE	$\sqrt{MSE}$	ошибка в масштабе целевой переменной
MAE	$\frac{1}{N}\sum\lvert y_i - \hat{y}_i\rvert$	устойчивее к выбросам
R^2	$1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$	доля объяснённой вариации

Практический тезис

Выбор метрики должен соответствовать цене ошибки. Если крупные ошибки особенно нежелательны, RMSE может быть информативнее MAE.

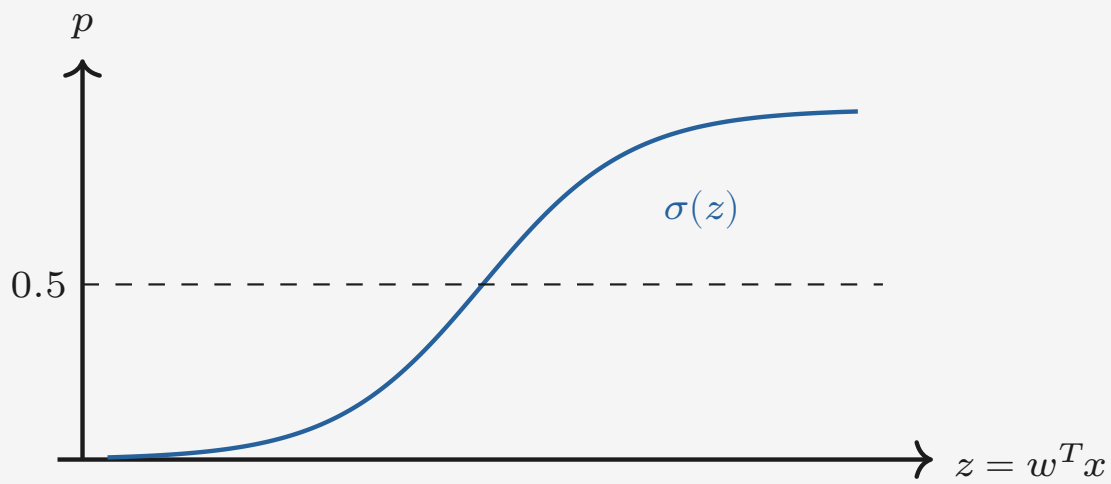


Логистическая регрессия

Бинарный случай

$$p(y = 1 \mid x) = \frac{1}{1 + \exp(-w^T x)}.$$

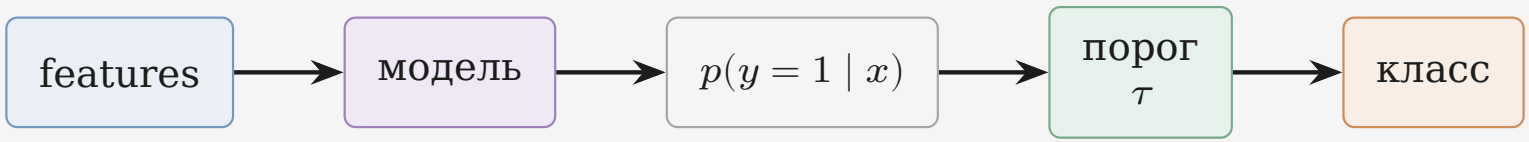
Модель оценивает вероятность принадлежности объекта к положительному классу.



Класс часто выбирается порогом: если $p \geq \tau$, объект относится к классу 1.

Порог классификации

Вероятностная модель и правило принятия решения — не одно и то же.



Пример

В медицинской диагностике или обнаружении фродов порог может быть специально снижен, чтобы увеличить recall, даже если возрастает число ложноположительных.



Матрица ошибок

		Предсказание	
		Positive	Negative
Истина	Positive	**TP**	**FN**
	Negative	**FP**	**TN**

- **TP**: положительный объект найден правильно;
- **FP**: модель ошибочно объявила объект положительным;
- **FN**: модель пропустила положительный объект;
- **TN**: отрицательный объект отвергнут правильно.

Все основные метрики классификации — это разные способы агрегировать четыре числа матрицы ошибок.



Precision, recall, F1

$$Precision = \frac{TP}{TP + FP}, \qquad Recall = \frac{TP}{TP + FN}.$$

$$F_1 = 2\frac{Precision \cdot Recall}{Precision + Recall}.$$

Precision

Из всех объектов, которые модель назвала положительными, какая доля действительно положительна?

Recall

Из всех действительно положительных объектов, какую долю модель нашла?

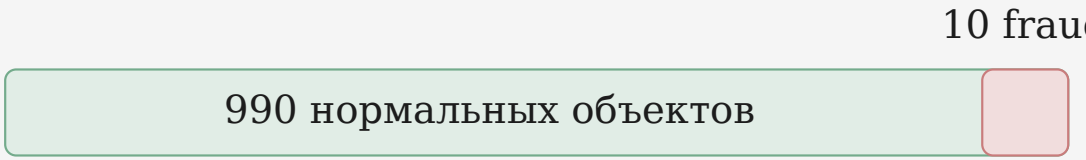


Почему accuracy недостаточно

Accuracy измеряет общую долю правильных ответов:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}.$$

Если положительный класс редок, модель «всегда отрицательно» может иметь высокую accuracy, но быть бесполезной.



Вывод

Для несбалансированных классов обязательно смотреть precision, recall, F1, ROC-AUC или PR-AUC, а не только accuracy.

Оценка качества в Spark ML

```
from pyspark.ml.evaluation import BinaryClassificationEvaluator

predictions = model.transform(test_df)

evaluator = BinaryClassificationEvaluator(
    labelCol="label",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

auc = evaluator.evaluate(predictions)
print(auc)
```

Инженерный смысл

Оценка качества — это не локальная операция над маленьким массивом. Если `predictions` велик, вычисление метрики также является распределённой Spark-задачей.

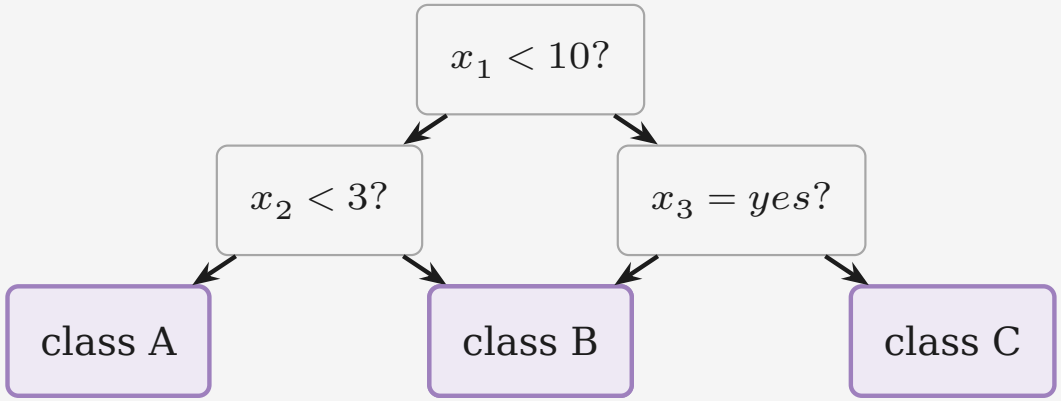


Раздел

Деревья решений и ансамбли



Дерево решений



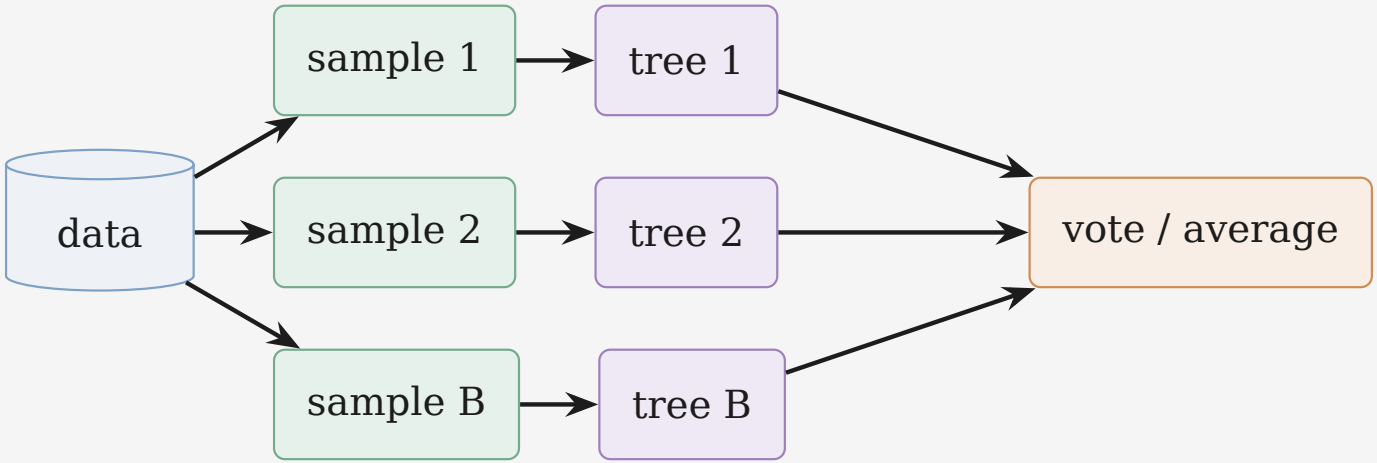
Определение

Дерево решений строит модель как последовательность правил разбиения пространства признаков. В каждом узле выбирается признак и порог, улучшающие чистоту дочерних подмножеств.

Деревья хорошо работают с нелинейными зависимостями и смешанными типами признаков, но отдельное дерево склонно к переобучению.



Bagging и Random Forest

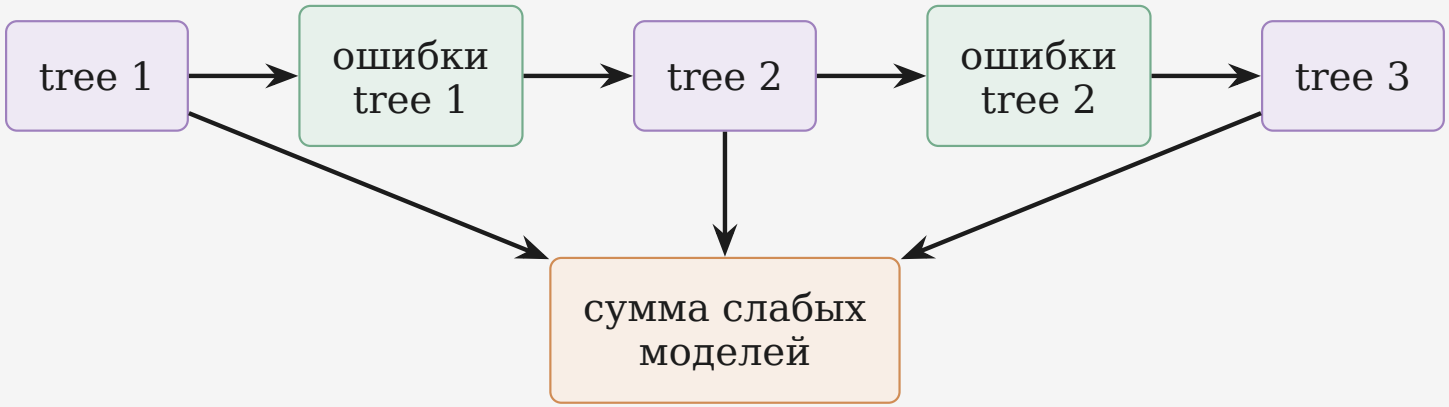


Идея

Bagging обучает много моделей на разных подвыборках и усредняет результат. **Random Forest** добавляет случайный выбор признаков при построении разбиений.



Boosting



Идея

Boosting строит ансамбль последовательно, где каждая следующая модель пытается исправить ошибки предыдущих. Поэтому boosting часто даёт высокое качество, но хуже параллелится, чем bagging.



Ансамбли с точки зрения вычислений

Случайный лес

- деревья относительно независимы;
- обучение хорошо распараллеливается;
- качество стабильно;
- модель может быть тяжёлой по памяти.

Градиентный бустинг

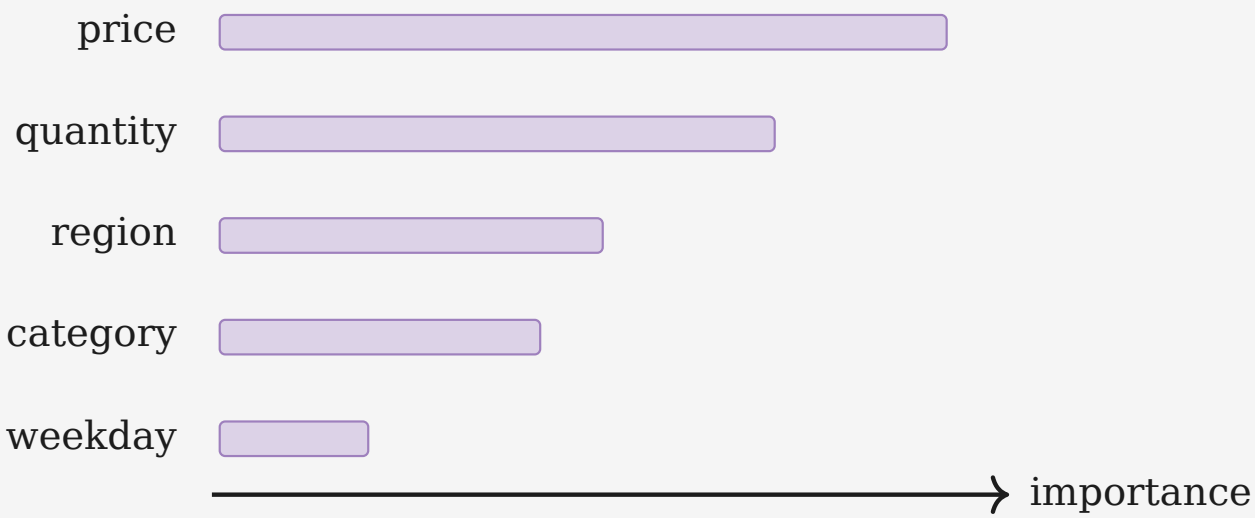
- модели зависят от предыдущих;
- меньше независимого параллелизма;
- часто выше качество;
- чувствителен к настройкам.

В Big Data-среде качество алгоритма всегда сопоставляется со стоимостью обучения и временем пересчёта модели.



Интерпретация важности признаков

Деревья и ансамбли часто позволяют оценивать важность признаков.



Осторожность

Feature importance не является строгим причинным объяснением. Она показывает вклад признака в работу конкретной модели на конкретных данных.

Раздел

Кластеризация



K-Means

Оптимизационная постановка

Алгоритм K-Means ищет центры кластеров $\mu_1, \dots, \mu_k$, минимизируя сумму квадратов расстояний до ближайшего центра:

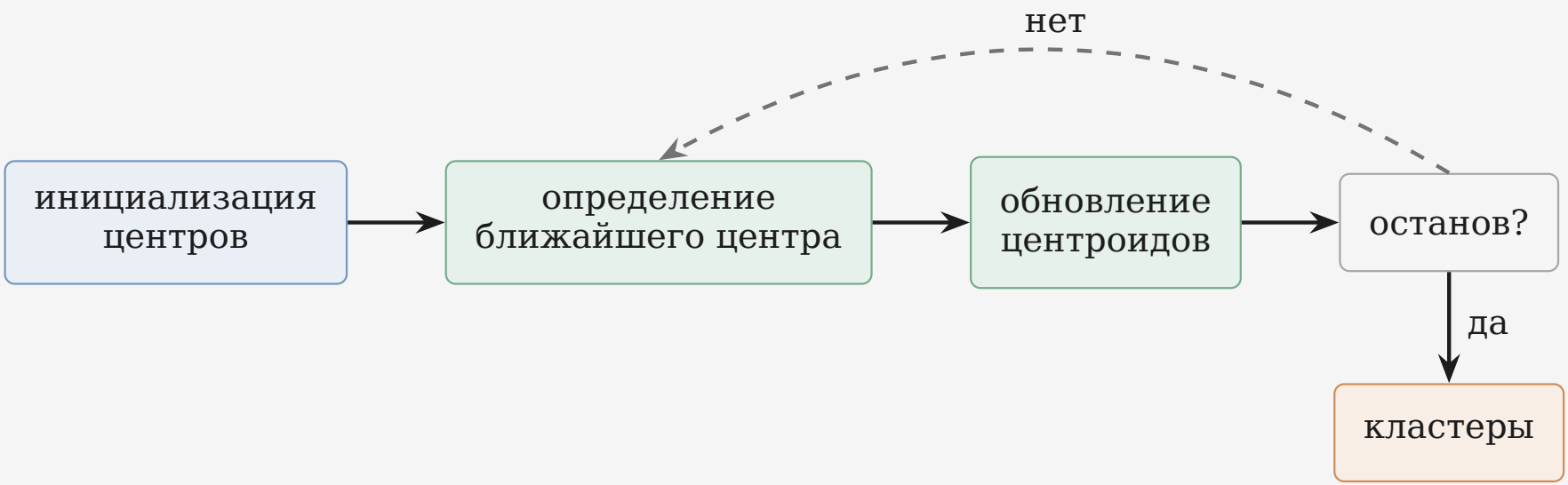
$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2.$$

Итерации:

1. назначить каждый объект ближайшему центру;
2. пересчитать центры как средние объектов кластера;
3. повторять до стабилизации или лимита итераций.



Схема итерации K-Means



Spark-смысл

Назначение объектов к центрам распараллеливается по партициям, но пересчёт центров требует глобальной агрегации.



Почему нормализация критична для K-Means

K-Means использует расстояние. Если один признак измеряется в тысячах, а другой в долях, первый будет доминировать.



Правило

Перед кластеризацией на расстояниях признаки обычно масштабируются. Иначе геометрия пространства будет определяться единицами измерения, а не структурой данных.



Силуэт

Идея метрики

Silhouette сравнивает среднее расстояние объекта до своего кластера и до ближайшего чужого кластера:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

- $a(i)$ — насколько объект далёк от объектов своего кластера;
- $b(i)$ — насколько объект близок к соседнему кластеру;
- значение ближе к 1 указывает на хорошее размещение объекта.

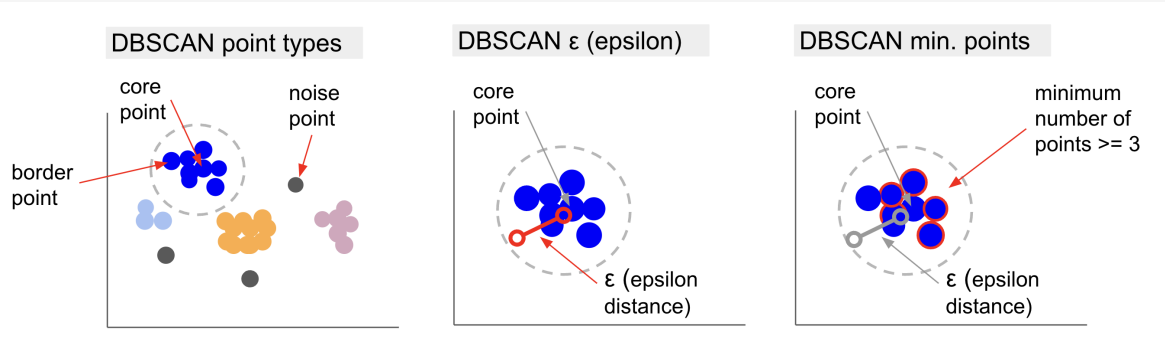
Это внутренняя метрика, которая не требует истинных меток классов.



Идея DBSCAN

Идея

DBSCAN группирует области высокой плотности и способен выделять шумовые точки, не требуя заранее задавать число кластеров.



Сильные стороны

- кластеры произвольной формы;
- явное понятие шума;
- не нужен параметр k.

Ограничения

- чувствительность к ε;
- проблемы в высокой размерности;
- распределённая реализация сложнее K-Means.

Кластеризация как аналитика

Кластеры не являются заранее существующими классами. Это результат выбранного пространства признаков, метрики расстояния и алгоритма.

Практический вопрос

После кластеризации нужно ответить не только «какой ситуэт», но и «можно ли интерпретировать группы и использовать их для решения задачи?»

Например, сегменты клиентов должны быть описаны понятным языком:

- частые покупатели с высоким средним чеком;
- редкие покупатели сезонных товаров;
- новые клиенты с негативными отзывами;
- клиенты с высокой вероятностью оттока.



Раздел

Кросс-валидация и подбор гиперпараметров



Параметры и гиперпараметры

Параметры модели

Обучаются по данным.

$$w_0, w_1, \dots, w_p$$

Например веса линейной модели.

Гиперпараметры

Задаются до обучения или выбираются внешней процедурой.

$$\lambda,\ depth,\ k,\ maxIter$$

Например регуляризация, глубина дерева, число кластеров.

Подбор гиперпараметров означает многократное обучение моделей и сравнение качества на валидации.

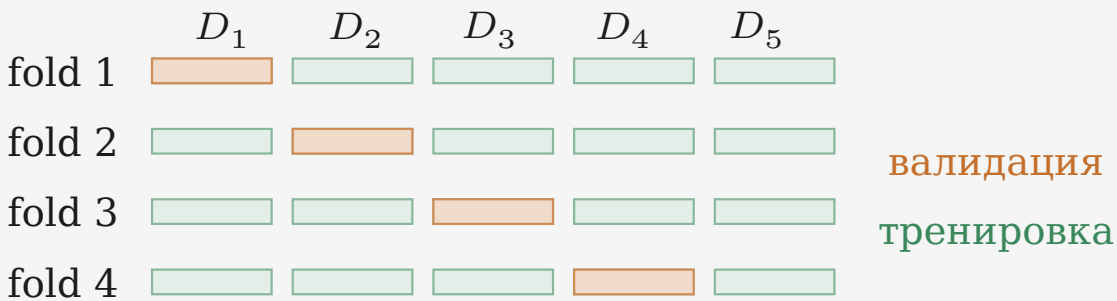


Кросс-валидация

Идея

Данные делятся на k частей. Каждая часть один раз становится валидацией, остальные используются для обучения:

$$D \to D_1, D_2, \dots, D_k.$$



Цена кросс-валидации

Если проверяется m комбинаций гиперпараметров и используется k-fold кросс-валидации, число обучений равно

$$N_{trainings} = m \cdot k.$$

Если каждая модель обучается T минут, то грубая оценка времени:

$$T_{total} \approx m \cdot k \cdot T,$$

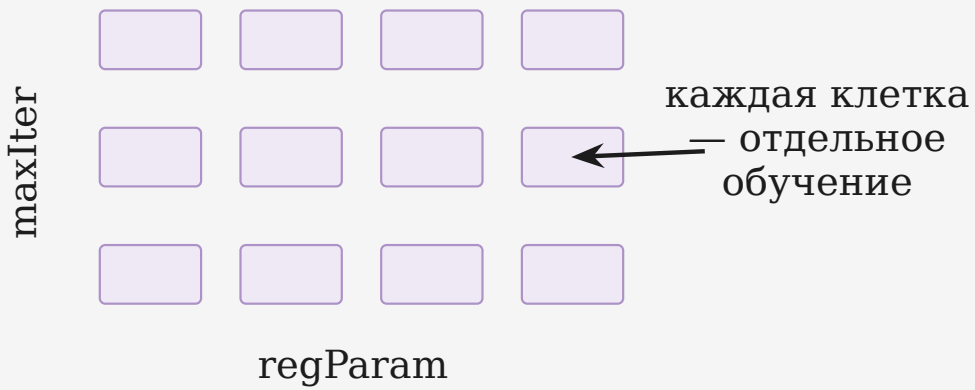
без учёта очередей, перемешивания и повторного чтения данных.

Big Data-следствие

В больших данных полный поиск по сетке быстро становится вычислительно неприемлемым. Нужно сокращать сетку, использовать разделение на тренировку и валидацию, отбор или более экономные стратегии поиска.



Поиск по сетке



Определение

Поиск по сетке перебирает заранее заданную сетку значений гиперпараметров и выбирает комбинацию с лучшей валидационной метрикой.



CrossValidator в Spark ML

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

param_grid = (ParamGridBuilder()
    .addGrid(lr.regParam, [0.01, 0.1, 1.0])
    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0])
    .build())

evaluator = BinaryClassificationEvaluator(metricName="areaUnderROC")

cv = CrossValidator(
    estimator=pipeline,
    estimatorParamMaps=param_grid,
    evaluator=evaluator,
    numFolds=3
)

cv_model = cv.fit(train_df)
```



TrainValidationSplit как компромисс

Идея

`TrainValidationSplit` не выполняет k-fold кросс-валидацию. Он один раз делит данные на тренировку и валидацию, затем сравнивает гиперпараметры.

Плюсы

- дешевле кросс-валидация;
- проще интерпретировать;
- подходит для больших наборов данных.

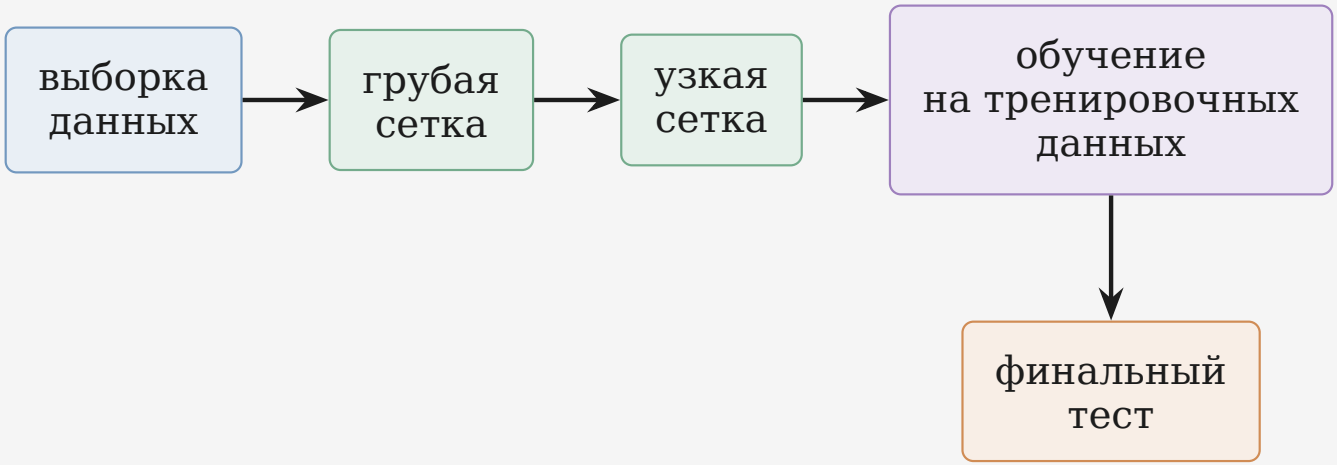
Минусы

- сильнее зависит от конкретного разбиения;
- менее устойчивая оценка;
- требует аккуратного выбора валидации.



Практическая стратегия подбора

В условиях ограниченных ресурсов разумная стратегия обычно ступенчатая.



Правило курса

Не начинать с тяжёлой кросс-валидации на полном наборе данных. Сначала проверить поток обработки, признаки и порядок величин метрик на ограниченной выборке.

Раздел

Масштабируемость ML



scikit-learn и Spark ML

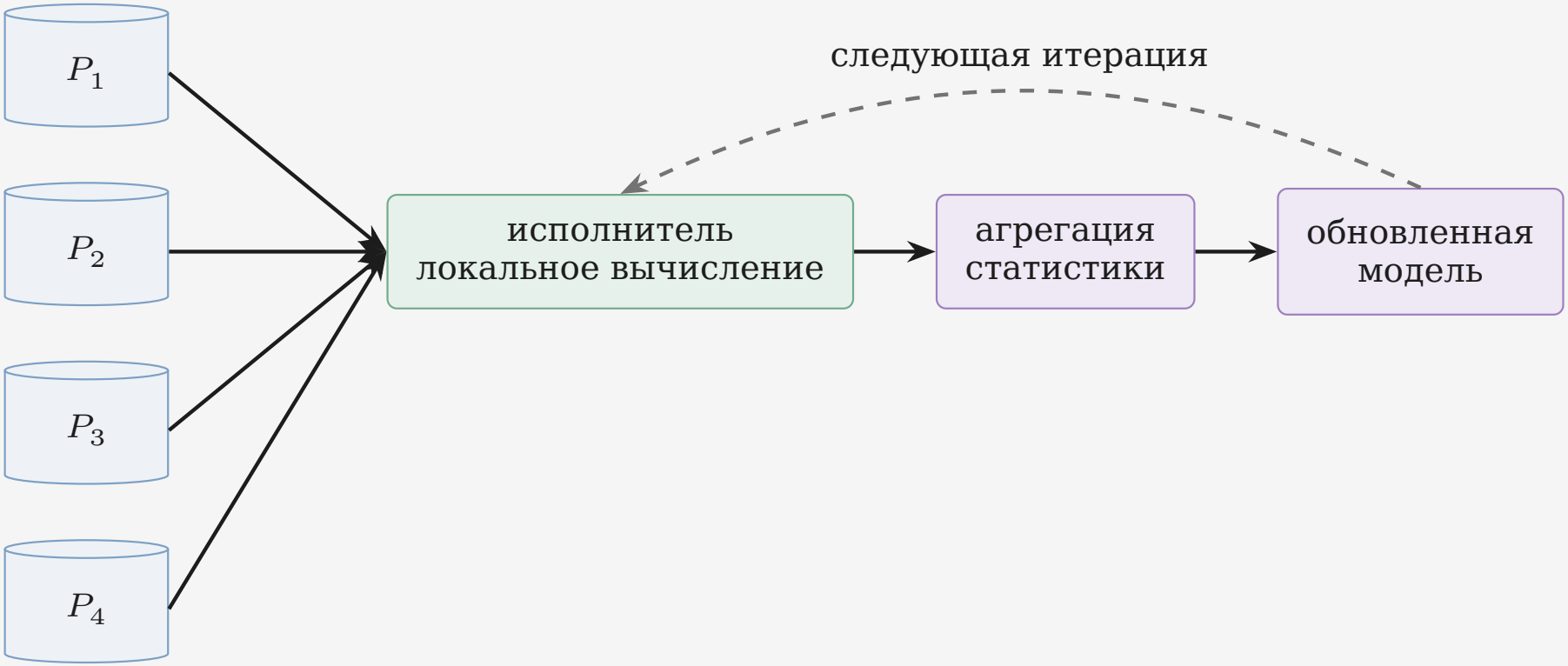
Критерий	scikit-learn	Spark ML
Вычисления	один процесс / один узел	распределённые исполнители
Данные	массивы в памяти	DataFrame по партициям
API	модель-оценщик	поток вычислений
Сильная сторона	быстрый запуск	большие таблицы
Ограничение	ОЗУ узла	число разделений

Вывод

Spark ML нужен не потому, что формулы регрессии становятся другими, а потому, что данные и вычисления перестают быть локальными.



Модель распределённого обучения



Смысл

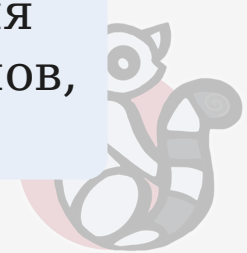
Каждый исполнит вычисляет локальные статистики по своим партициям, затем Spark агрегирует результаты, обновляет модель и повторяет процедуру.

Узкие места Spark ML

- **Перемешивание**: перераспределение данных при объединении, группировке, некоторых алгоритмах.
- **Перекос**: неравномерное распределение данных по партициям.
- **Повторяемое сканирование**: многократное чтение одних и тех же данных при обучении и подборе параметров.
- **Высокая кардинальность**: слишком много категорий после one-hot кодирования.
- **Сбор**: перенос больших результатов на драйвер.
- **Неправильный кэш**: кэширование слишком больших или редко используемых данных.

Инженерный критерий

Оптимизация ML потока обработки начинается с понимания плана выполнения, размера данных, числа партиций и этапов, которые пересчитываются многократно.



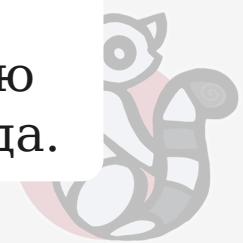
Когда Spark ML оправдан

Spark ML особенно уместен, если:

- данные уже находятся в Spark/Data Lake/DWH-контуре;
- объём данных превышает удобный режим одного узла;
- требуется массовое построение признаков через Spark SQL;
- модель должна применяться к большому числу объектов;
- pipeline должен быть частью регулярной ETL/ELT-процедуры.

Когда Spark может быть избыточен

Если данных мало, а модель сложная и нетривиальная, быстрее и проще использовать локальные библиотеки: scikit-learn, PyTorch, CatBoost, XGBoost. Spark имеет высокую сложность написания сценария, которая окупается не всегда.



Раздел

Итоги



Ключевые понятия

- supervised / unsupervised learning;
- regression, classification, clustering;
- train / validation / test;
- data leakage;
- feature engineering;
- sparse / dense vector;
- Transformer, Estimator, Model;
- Spark ML Pipeline;
- linear regression;
- logistic regression;
- precision, recall, F1;
- decision tree, random forest, boosting;
- K-Means, silhouette;
- cross-validation и grid search.



Контрольные вопросы

1. Почему preprocessing должен быть частью pipeline, а не отдельным действием до split?
2. Чем `Transformer` отличается от `Estimator`?
3. Почему accuracy может быть плохой метрикой для fraud detection?
4. В чём вычислительная цена k-fold cross-validation?
5. Почему K-Means чувствителен к масштабу признаков?
6. Чем bagging отличается от boosting с точки зрения параллелизма?
7. Когда Spark ML предпочтительнее scikit-learn, а когда избыточен?



Спасибо за внимание

